

CSS Grid - Nested Grids

Complete Guide: From Basics to Advanced

Contents

1	Introduction	2
1.1	What are Nested Grids?	2
1.2	Why Use Nested Grids?	2
1.3	Nested Grids vs. Subgrid	2
2	Core Concepts	3
2.1	Basic Nested Grid Structure	3
2.1.1	HTML Structure	3
2.1.2	CSS Structure	3
2.2	Key Rules	4
3	Common Patterns	5
3.1	Pattern 1: Dashboard with Cards	5
3.2	Pattern 2: Blog Layout with Sidebar	6
3.3	Pattern 3: Product Grid with Details	7
3.4	Pattern 4: Admin Dashboard Layout	8
4	Responsive Nested Grids	10
4.1	Mobile-First Approach	10
4.2	Complex Responsive Example	11
5	Advanced Techniques	13
5.1	Grid Auto-Flow with Nesting	13
5.2	Using CSS Grid Lines Names	13
5.3	Combining Grid with Flexbox	14
5.4	Grid with CSS Grid Subgrid (Modern)	15
6	Common Patterns & Use Cases	16
6.1	E-Commerce Product Layout	16
6.2	Blog Comment Section	17
7	Troubleshooting	19
7.1	Issue 1: Child Grid Doesn't Respect Parent Gap	19
7.2	Issue 2: Child Grid Overflows Container	19
7.3	Issue 3: Nested Grid Alignment Issues	20

8	Best Practices	21
8.1	Performance Tips	21
8.2	Maintainability	21
8.3	Accessibility	21
8.4	Browser Support	22
9	Summary: Key Takeaways	23
9.1	When to Use Nested Grids	23
9.2	When NOT to Use Nested Grids	23
9.3	Quick Reference	23
9.4	Key Rules	24

Chapter 1

Introduction

1.1 What are Nested Grids?

Nested grids are CSS Grid containers placed inside other CSS Grid containers. This allows you to create complex, multi-level layouts where grid items can themselves be grids, enabling powerful and flexible page structures.

Key Point: A grid item can be a grid container itself. This creates a hierarchy of grids without affecting the parent grid's layout.

1.2 Why Use Nested Grids?

- Create complex, hierarchical layouts
- Organize content at different levels
- Maintain responsive design at multiple levels
- Keep code modular and maintainable
- Enable independent control of sub-sections

1.3 Nested Grids vs. Subgrid

Nested Grids	Subgrid (CSS Grid Level 2)
Child grid is independent	Child inherits parent's grid lines
Can have different columns/rows	Aligns with parent grid
Wider browser support	Limited browser support
More flexible	Better alignment control
Simpler to understand	Advanced technique

Chapter 2

Core Concepts

2.1 Basic Nested Grid Structure

2.1.1 HTML Structure

```
<div class="parent-grid">
  <div class="grid-item">
    <div class="child-grid">
      <div class="child-item">Item 1</div>
      <div class="child-item">Item 2</div>
      <div class="child-item">Item 3</div>
    </div>
  </div>
</div>
```

2.1.2 CSS Structure

```
/* Parent Grid */
.parent-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}

/* Grid Item (also a container) */
.grid-item {
  background: lightblue;
  padding: 10px;
}

/* Child Grid - nested inside grid-item */
.child-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 10px;
}

/* Child Items */
```

```
.child-item {  
  background: white;  
  padding: 15px;  
  border: 1px solid #ddd;  
}
```

2.2 Key Rules

1. A grid item can contain any content, including a grid
2. The child grid is independent of the parent grid
3. Grid gap only affects that specific grid level
4. Each grid can have different column/row configurations
5. Nesting depth is unlimited

Chapter 3

Common Patterns

3.1 Pattern 1: Dashboard with Cards

```
/* Main dashboard grid */
.dashboard {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 20px;
  padding: 20px;
}

/* Each card is a grid item */
.card {
  background: white;
  border-radius: 8px;
  padding: 20px;
}

/* Card spans 2 columns */
.card-large {
  grid-column: span 2;
}

/* Card contents are nested grid */
.card-content {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 15px;
}

.card-stat {
  background: #f0f0f0;
  padding: 10px;
  border-radius: 4px;
}
```

Result: Dashboard with cards containing internal grid layouts.

3.2 Pattern 2: Blog Layout with Sidebar

```
/* Main layout grid */
.blog-layout {
  display: grid;
  grid-template-columns: 1fr 300px;
  gap: 30px;
}

/* Main content area */
.main-content {
  display: grid;
  grid-template-columns: 1fr;
  gap: 30px;
}

/* Each article is also a grid */
.article {
  display: grid;
  grid-template-columns: 200px 1fr;
  gap: 20px;
  background: white;
  padding: 20px;
}

.article-image {
  width: 200px;
  height: 200px;
}

.article-text {
  display: grid;
  grid-template-rows: auto auto auto;
  gap: 10px;
}

.article-title { font-size: 20px; font-weight: bold; }
.article-meta { color: #666; font-size: 12px; }
.article-summary { line-height: 1.6; }

/* Sidebar with nested grid */
.sidebar {
  display: grid;
  grid-template-columns: 1fr;
  gap: 20px;
}

.widget {
  background: white;
  padding: 15px;
  border-radius: 4px;
}
```



```
.widget-list {
  display: grid;
  gap: 10px;
}

.widget-item {
  padding: 8px;
  border-left: 3px solid #007bff;
  padding-left: 10px;
}
```

Result: Blog with article cards and sidebar widgets, each with nested grids.

3.3 Pattern 3: Product Grid with Details

```
/* Product grid */
.products {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  gap: 20px;
}

/* Each product card is a grid */
.product-card {
  display: grid;
  grid-template-rows: 200px auto auto;
  gap: 15px;
  background: white;
  border-radius: 8px;
  overflow: hidden;
}

.product-image {
  width: 100%;
  height: 200px;
  object-fit: cover;
}

/* Product info section */
.product-info {
  padding: 15px;
  display: grid;
  gap: 10px;
}

.product-name {
  font-weight: bold;
  font-size: 16px;
}

.product-rating {
  display: grid;
  grid-template-columns: auto 1fr;
}
```

```
    gap: 10px;
    align-items: center;
}

.stars { color: #ff9800; }
.reviews { color: #999; font-size: 12px; }

/* Price section nested */
.product-price {
    display: grid;
    grid-template-columns: auto auto;
    gap: 10px;
    align-items: center;
}

.current-price {
    font-size: 18px;
    font-weight: bold;
    color: #e74c3c;
}

.original-price {
    text-decoration: line-through;
    color: #999;
}
```

Result: Product showcase with nested grid structure for images, info, and pricing.

3.4 Pattern 4: Admin Dashboard Layout

```
/* Main layout */
.admin-dashboard {
    display: grid;
    grid-template-columns: 250px 1fr;
    grid-template-rows: 60px 1fr;
    min-height: 100vh;
}

.header { grid-column: 1 / -1; }
.sidebar { grid-row: 2; }
.content { grid-row: 2; }

/* Header as grid */
.header {
    display: grid;
    grid-template-columns: 1fr auto auto;
    gap: 20px;
    align-items: center;
    padding: 0 20px;
}

/* Sidebar nested grid */
.sidebar {
```

```
    display: grid;
    grid-template-rows: auto 1fr;
    gap: 20px;
    padding: 20px;
}

.sidebar-menu {
    display: grid;
    gap: 10px;
}

.menu-item {
    padding: 10px;
    cursor: pointer;
    border-radius: 4px;
}

/* Main content area */
.content {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
    padding: 20px;
}

/* Stats cards */
.stat-card {
    display: grid;
    grid-template-rows: auto 1fr;
    gap: 10px;
    background: white;
    padding: 20px;
    border-radius: 8px;
}

.stat-label { font-size: 12px; color: #666; }
.stat-value { font-size: 24px; font-weight: bold; }

/* Charts section spans 2 columns */
.chart-section {
    grid-column: span 2;
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 20px;
}
```

Result: Admin dashboard with hierarchical grid layout.

Chapter 4

Responsive Nested Grids

4.1 Mobile-First Approach

```
/* Mobile: Single column everywhere */
.dashboard {
  display: grid;
  grid-template-columns: 1fr;
  gap: 20px;
}

.card {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
}

.card-content {
  grid-template-columns: 1fr;
}

/* Tablet: 2 columns */
@media (min-width: 768px) {
  .dashboard {
    grid-template-columns: repeat(2, 1fr);
  }

  .card-large {
    grid-column: span 1;
  }

  .card-content {
    grid-template-columns: 1fr 1fr;
  }
}

/* Desktop: 4 columns */
@media (min-width: 1024px) {
  .dashboard {
    grid-template-columns: repeat(4, 1fr);
  }
}
```

```
    }

    .card-large {
        grid-column: span 2;
    }
}
```

4.2 Complex Responsive Example

```
/* Base mobile layout */
.layout {
    display: grid;
    grid-template-columns: 1fr;
    grid-template-rows: auto auto auto;
}

.header { grid-row: 1; }
.sidebar { grid-row: 2; display: none; }
.main { grid-row: 3; }

.main {
    display: grid;
    grid-template-columns: 1fr;
}

.article {
    display: grid;
    grid-template-columns: 1fr;
}

/* Tablet: Sidebar appears */
@media (min-width: 768px) {
    .layout {
        grid-template-columns: 1fr;
        grid-template-rows: auto 1fr;
    }

    .header { grid-row: 1; grid-column: 1 / -1; }
    .sidebar {
        grid-row: 2;
        grid-column: 1;
        display: grid;
    }
    .main {
        grid-row: 2;
        grid-column: 2;
    }

    .main {
        grid-template-columns: repeat(2, 1fr);
    }
}
```

```
/* Desktop: Full layout */
@media (min-width: 1200px) {
  .main {
    grid-template-columns: repeat(3, 1fr);
  }

  .article {
    grid-column: span 2;
  }
}
```

Chapter 5

Advanced Techniques

5.1 Grid Auto-Flow with Nesting

```
/* Dense packing in nested grid */
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  grid-auto-flow: dense;
  gap: 10px;
}

.gallery-item {
  background: #f0f0f0;
  height: 200px;
}

.gallery-item.tall {
  grid-row: span 2;
}

.gallery-item.wide {
  grid-column: span 2;
}

/* Nested grid inside gallery item */
.gallery-item.featured {
  grid-column: span 2;
  grid-row: span 2;
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 10px;
}
```

5.2 Using CSS Grid Lines Names

```
/* Parent grid with named lines */
.layout {
```

```
    display: grid;
    grid-template-columns: [sidebar-start] 250px [sidebar-end content-start] 1fr [content-end];
    grid-template-rows: [header-start] 60px [header-end content-start] 1fr [content-end];
  }

.header { grid-column: sidebar-start / content-end; }
.sidebar { grid-column: sidebar-start / sidebar-end; }
.main { grid-column: content-start / content-end; }

/* Nested grid with its own named lines */
.main {
  display: grid;
  grid-template-columns: [cards-start] repeat(auto-fit, minmax(250px, 1fr)) [cards-end];
  gap: 20px;
}

.card {
  grid-column: cards-start / cards-end;
  display: grid;
  grid-template-columns: [image-start] 100px [image-end content-start] 1fr [content-end];
}
```

5.3 Combining Grid with Flexbox

```
/* Parent grid */
.container {
  display: grid;
  grid-template-columns: 1fr 300px;
  gap: 20px;
}

/* Nested grid with flexbox items */
.content {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 15px;
}

/* Cards use flexbox for vertical layout */
.card {
  display: flex;
  flex-direction: column;
  gap: 10px;
  padding: 15px;
}

.card-header {
  display: flex;
```



```
    justify-content: space-between;
    align-items: center;
}

/* Sidebar with flexbox nested in grid */
.sidebar {
    display: flex;
    flex-direction: column;
    gap: 20px;
}

.widget {
    display: grid;
    grid-template-columns: 1fr;
    gap: 10px;
}
```

5.4 Grid with CSS Grid Subgrid (Modern)

```
/* Parent grid with named rows */
.form {
    display: grid;
    grid-template-columns: 150px 1fr;
    grid-template-rows: auto auto auto auto;
    gap: 15px;
}

/* Child grid inherits parent's columns and rows */
.form-group {
    display: grid;
    grid-column: span 2;
    grid-template-columns: subgrid;
    gap: inherit;
}

.label {
    text-align: right;
}

.input {
    width: 100%;
}

/* Browser Support: Chrome 71+, Firefox 71+, Safari 16+ */
```

Chapter 6

Common Patterns & Use Cases

6.1 E-Commerce Product Layout

```
/* Main product page */
.product-page {
  display: grid;
  grid-template-columns: 1fr 350px;
  gap: 40px;
  padding: 30px;
}

/* Image gallery grid */
.gallery {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}

.gallery-main {
  grid-column: 1 / -1;
  height: 400px;
}

.gallery-thumb {
  height: 100px;
}

/* Product details sidebar */
.details {
  display: grid;
  grid-template-columns: 1fr;
  gap: 20px;
}

.price-section {
  display: grid;
  grid-template-columns: auto 1fr;
  gap: 10px;
}
```

```
.variants {
  display: grid;
  gap: 15px;
}

.variant-group {
  display: grid;
  gap: 8px;
}

.variant-options {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(40px, 1fr));
  gap: 8px;
}

.variant-button {
  aspect-ratio: 1;
  cursor: pointer;
  border: 2px solid #ddd;
  border-radius: 4px;
}
```

6.2 Blog Comment Section

```
/* Comments container */
.comments {
  display: grid;
  gap: 20px;
}

/* Each comment is a grid */
.comment {
  display: grid;
  grid-template-columns: 60px 1fr;
  gap: 15px;
  padding: 15px;
  border-left: 3px solid #ddd;
}

.comment-avatar {
  width: 60px;
  height: 60px;
  border-radius: 50%;
}

.comment-body {
  display: grid;
  grid-template-columns: 1fr auto;
  gap: 10px;
}
```

```
.comment-header {
  display: grid;
  gap: 5px;
}

.comment-author {
  font-weight: bold;
}

.comment-date {
  color: #999;
  font-size: 12px;
}

.comment-actions {
  display: grid;
  grid-template-columns: auto auto;
  gap: 10px;
}

/* Nested replies */
.replies {
  display: grid;
  gap: 15px;
  margin-top: 15px;
  padding-left: 20px;
}

.reply {
  display: grid;
  grid-template-columns: 40px 1fr;
  gap: 10px;
}
```

Chapter 7

Troubleshooting

7.1 Issue 1: Child Grid Doesn't Respect Parent Gap

Problem: Gap from parent grid affects child grid placement.

Solution: Child grid has its own independent gap value.

```
/* Parent gap only affects parent items */
.parent {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px; /* Only between parent items */
}

/* Child gap is separate */
.child {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 10px; /* Only between child items */
}
```

7.2 Issue 2: Child Grid Overflows Container

Problem: Content inside nested grid overflows the grid item.

Solution: Add overflow handling or adjust grid sizing.

```
/* Option 1: Allow overflow */
.card {
  overflow: visible;
}

/* Option 2: Clip overflow */
.card {
  overflow: hidden;
}

/* Option 3: Add min-width 0 */
.card {
  min-width: 0;
  display: grid;
}
```

```
}

/* Option 4: Use minmax */
.card {
  display: grid;
  grid-template-columns: minmax(0, 1fr);
}
```

7.3 Issue 3: Nested Grid Alignment Issues

Problem: Items in nested grid don't align properly.

Solution: Use grid alignment properties.

```
/* Align items in child grid */
.child-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  align-items: center;      /* Vertical alignment */
  justify-items: center;    /* Horizontal alignment */
}

/* Or align individual items */
.child-item {
  align-self: center;
  justify-self: center;
}
```

Chapter 8

Best Practices

8.1 Performance Tips

1. Don't nest grids unnecessarily - sometimes flexbox is better
2. Keep nesting depth reasonable (3-4 levels max)
3. Use media queries to adapt nested layouts at different breakpoints
4. Avoid deeply nested responsive calculations

8.2 Maintainability

```
/* GOOD: Clear structure */
.layout { display: grid; }
.layout-sidebar { display: grid; }
.card { display: grid; }
.card-content { display: grid; }

/* BAD: Confusing nesting */
.wrapper > .container > .item > .sub > .content {
  display: grid;
}
```

8.3 Accessibility

- Keep logical HTML structure
- Don't use nested grids to reorder critical content
- Use semantic HTML inside nested grids
- Test with keyboard navigation
- Ensure color contrast in nested sections

8.4 Browser Support

- Nested grids: All modern browsers (Chrome 57+, Firefox 52+, Safari 10.1+)
- CSS Grid Subgrid: Chrome 71+, Firefox 71+, Safari 16+
- Fallbacks: Use flexbox for older browsers

Chapter 9

Summary: Key Takeaways

9.1 When to Use Nested Grids

- Complex multi-level layouts
- Dashboard with cards and internal layouts
- Product cards with nested content
- Blog layouts with sidebars and article sections
- Admin panels with hierarchical sections

9.2 When NOT to Use Nested Grids

- Simple linear layouts (use flexbox)
- When alignment must sync across levels (use subgrid)
- If nesting depth exceeds 3-4 levels

9.3 Quick Reference

```
/* Basic nested grid template */
.parent-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}

.child-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 10px;
}

/* Responsive nested grids */
@media (max-width: 768px) {
```

```
.parent-grid {  
  grid-template-columns: 1fr;  
}  
  
.child-grid {  
  grid-template-columns: 1fr;  
}  
}
```

9.4 Key Rules

1. Child grids are completely independent
2. Each grid level can have different column/row setup
3. Nesting depth is unlimited (but keep it reasonable)
4. Gap only affects that specific grid level
5. Use subgrid when you need alignment across levels
6. Always maintain responsive layouts at each level
7. Test deeply nested layouts thoroughly
8. Combine with flexbox when appropriate